



follow all the instructions carefully and guide me through this step by step Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account. THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!! Duration 60 minutes Software Requirements e Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc) Instructions: Go to the repo assigned to you
https://github.com/SETAPESU26/48_whack-a-mole
Go through the ReadMe file to understand the deliverables
Clone or Fork the repo
Run the Python based code (A game/ A Puzzle)
Record 10 seconds of video before making any changes.
Write prompts and fix the broken code
Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes. 9. Push the code to your repo.
10. Have Separate Commits for each task
11. Do not raise PR to the main repo (SETAPESU26) NQUPWNP
Deliverables:
Upload the following to your Lab repo under Lab-4 folder,
a. Videos (before and after) b. Updated Code c. Chat history exported as a doc/pdf

09/10/2026, 09:26:38



Thinking

- Searched 6 websites



✓ Worked for 6s

We'll do this **one step at a time, on your Kali Linux system**, and make sure you understand what you're doing rather than blindly copying commands.

Your assignment has four main goals:

1. Get the assigned Whack-a-Mole repository onto your computer and run the original game.
2. Record a 10-second video **before making any changes**.
3. Use a vibe-coding tool to fix the bugs and implement every feature required by the README, with a separate Git commit for each task.
4. Record the updated game, push your code to your own GitHub repository, and submit the videos, code, and exported chat history in your Lab-4 folder.

You have 60 minutes, so we'll prioritize the required deliverables and verify each step before moving forward.

Step 1 — Open the assigned repository

Your assigned repository is:

First, open it in your browser and check these two things:

- **README.md:** find the list of bugs, required fixes, and features you must implement.
- **Fork or clone:** you'll need your own repository under your personal GitHub account. Do not push changes to `SETAPESU26`.

I couldn't verify the contents of your exact assigned repository from the search results, so I won't guess which bugs or features it requires. We'll use the actual README and code before writing prompts.

Step 2 — Fork the repository to your account

Do this before changing any code.

1. On the GitHub page, sign in to your **personal GitHub account**.
2. Click **Fork** near the top-right of the repository.



3. Choose your personal account as the owner.
4. Keep the repository name `48_whack-a-mole` , unless GitHub requires a different name.
5. Click **Create fork**.

Your new repository should have a URL similar to:

```
https://github.com/YOUR-USERNAME/48_whack-a-mole
```

Replace `YOUR-USERNAME` with your actual GitHub username.

Important: Do not click Pull requests or create a PR to the college repository. Your assignment specifically says not to raise a PR to `SETAPESU26` .

Your turn

Complete Step 1 and Step 2 first. Then tell me:

- Did the repository open successfully?
- Were you able to fork it?
- Can you see the README's required features?

If the repository is private, you get a 404, or the Fork button is missing, tell me exactly what you see.

Once we confirm that, I'll guide you through cloning it on Kali, checking Python dependencies, running the original game, and recording your **before** video before we touch the code

09/10/2026, 09:26:41 · gpt-6





Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Refine Collision Detection

A single click can sometimes award two points instead of one, when two neighboring moles are both up and the click lands near the border between their holes. Investigate and enhance click-hit accuracy.

Task 2: Implement Game Over Condition

Add a screen that displays the final score once the round timer reaches zero, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option

After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard mole speed/spawn rate), or exit.

Task 4: Add Sound Feedback

Add basic sound effects for a successful whack, a missed click, and the round ending.

Expected Behavior

- Moles pop up at random holes for a short time and go back down if not whacked
- Clicking directly on an active mole scores a point and sends it back down immediately
- Clicking an empty hole (or missing entirely) does not score
- A countdown timer and the current score are visible at all times
- The round ends when the timer reaches zero

Folder Structure

```
whack-a-mole-main/  
├── main.py  
├── requirements.txt  
├── game/  
│   ├── game_engine.py  
│   └── hole.py  
└── README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

these are the instructions in readme.md

09/10/2026, 09:28:18



Perfect. Now we know the **exact four tasks** from your README. We'll complete them individually, using an LLM for suggestions and reviewing the generated code before committing each change.

Our 60-minute plan

****1. Fork, clone and run the original game**** 8 minutes · Verify Python dependencies and startup. ****2. Record the before video**** 3 minutes · Capture 10 seconds before any code changes. ****3. Fix bugs and add features**** 32 minutes · Four separate tasks and four commits. ****4. Test, record after and submit**** 17 minutes · Push to your fork and prepare deliverables.



The four tasks we'll implement

Commit	Task	What we need to verify
1	Collision detection	One click on one active mole gives at most one point.
2	Game-over screen	Timer reaches zero, final score appears, and the game waits for input.
3	Replay and difficulty	Easy, Medium and Hard restart the game at different mole speeds/spawn rates; exit is available.
4	Sound feedback	Different feedback for a successful hit, a missed click and the end of the round.

We'll inspect the existing code first, preserve the current project structure, and use a separate Git commit for each task. After each change, we'll review and test it rather than blindly accepting the LLM's output.

Step 1 - Fork and clone on Kali

If you haven't forked the repository yet, do that first:

[Open the assigned repository](#).

Then open a terminal on Kali Linux and run:

If you have already forked the repo, clone **your fork**, not the original college repository:

Replace `YOUR-USERNAME` with your GitHub username.

Now inspect the project:

These commands show your current directory, remote GitHub URLs, working-tree status, project files and README. Make sure `origin` points to your personal fork.

Before you continue

Run those commands and send me the output of:





```
**THIS THE OUPUT OF THE cmmnds ,tasya\@kali**  
**48_whack-a-mole** **main**  
➤ git remote -v  
ls -la  
origin [https://github.com/PES1UG24AM411/48_whack-a-mole.git]  
(https://github.com/PES1UG24AM411/48_whack-a-mole.git) (fetch)  
origin [https://github.com/PES1UG24AM411/48_whack-a-mole.git]  
(https://github.com/PES1UG24AM411/48_whack-a-mole.git) (push)  
**d**rwxrwxr-x tasya tasya 4.0 KB Fri Oct 9 09:31:03 2026 ** .**b>  
**d**rwxrwxr-x tasya tasya 4.0 KB Fri Oct 9 09:31:02 2026 ** ..**  
**d**rwxrwxr-x tasya tasya 4.0 KB Fri Oct 9 09:31:08 2026 ** .git**  
**d**rwxrwxr-x tasya tasya 4.0 KB Fri Oct 9 09:31:03 2026 ** game**  
.rw-rw-r-- tasya tasya 832 B Fri Oct 9 09:31:03 2026 main.py  
.rw-rw-r-- tasya tasya 2.7 KB Fri Oct 9 09:31:03 2026 README.md  
.rw-rw-r-- tasya tasya 7 B Fri Oct 9 09:31:03 2026 requirements.txt
```

09/10/2026, 09:32:34



Good! Your clone is successful, and your terminal shows you're inside `48_whack-a-mole` on the `main` branch.

One important detail: your `origin` is `PES1UG24AM411/48_whack-a-mole.git`. That appears to be your personal fork, which is what we want. We'll verify that you can push to it later.

Step 2 - Inspect the Python files



Before changing anything, we need to understand how the original game works and what dependencies it needs.

Copy and run these commands in your Kali terminal, one at a time.

1. Read the dependency file

2. Read the main game file

3. List the game files

4. Check Python and pip

Step 3 - Prepare to run the original game

Once you've inspected the files, create a virtual environment. This keeps the project's Python dependencies separate from Kali's system Python.

If that command succeeds, activate it:

Your terminal prompt should now show `(.venv)`.

Install the project's dependencies:

If creating the virtual environment fails with an error about `ensurepip` or `venv`, stop and send me the error instead of modifying Kali's system Python.

Step 4 - Run and record the original game

Do not edit any files yet. We must capture the original game before making changes.

Once the dependencies are installed, run:

If the game opens, use Kazam to record at least 10 seconds of gameplay showing the original behavior. Save the video somewhere safe, such as:

```
\~/Videos/whack-a-mole-before.mp4
```

If the game doesn't open, send me the complete error message.

Your next action

Run the four inspection commands first and send me their output, especially `main.py` and `requirements.txt`. We'll use that to choose the correct run command and identify the original bug without guessing.

09/10/2026, 09:32:37 · gpt-6





ok , the game waas executed, this is the main.py

```
"import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 500, 560
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Whack-a-Mole - Pygame
Version")

# Colors
GRASS_GREEN = (120, 170, 90)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(GRASS_GREEN)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

    pygame.display.flip()
    clock.tick(FPS)
```



```
pygame.quit()
```

```
if __name__ == "__main__":  
    main()
```

and these are the changes to be made are "Tasks to Complete"

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Refine Collision Detection

> A single click can sometimes award two points instead of one, when two neighboring moles are both up and the click lands near the border between their holes. Investigate and enhance click-hit accuracy.

Task 2: Implement Game Over Condition

> Add a screen that displays the final score once the round timer reaches zero, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option

> After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard mole speed/spawn rate), or exit.

Task 4: Add Sound Feedback

> Add basic sound effects for a successful whack, a missed click, and the round ending.

Expected Behavior

- Moles pop up at random holes for a short time and go back down if not whacked
- Clicking directly on an active mole scores a point and



sends it back down immediately

- Clicking an empty hole (or missing entirely) does not score
- A countdown timer and the current score are visible at all times
- The round ends when the timer reaches zero

Folder Structure

```

```
whack-a-mole-main/
├─ main.py
├─ requirements.txt
├─ game/
│ ├─ game_engine.py
│ └─ hole.py
└─ README.md"""
```

```

09/10/2026, 09:41:41



Great! Since the original game runs, **don't change any code yet**. We need to secure the before-video evidence first, then work through all four tasks with separate commits.

Your `main.py` shows that the game loop delegates its logic to `GameEngine` in `game/game_engine.py`. That's where we'll likely need to inspect the collision detection, timer, and game-state behavior.

Step 1 - Record the original game (10 seconds)

If you haven't recorded it yet, do this now.

1. Open Kazam.
2. Select screen recording.
3. Start recording, then play the original game for at least 10 seconds.



4. Try clicking near the border between two neighboring active moles to capture the double-scoring bug, if you can reproduce it.
5. Stop recording and save it as `whack-a-mole-before.mp4` in your Videos folder.

Verify the file exists:

If Kazam saves it under a different name or folder, use the actual path.

Step 2 - Inspect the actual game logic

We need to read the engine and hole classes before asking the coding assistant to change anything.

Run:

Then:

Also check the installed dependency:

These files will tell us how moles are positioned, how mouse clicks are processed, and whether a timer or scoring system already exists.

Step 3 - Start Task 1 using vibe coding

We'll use ChatGPT for the first code-review prompt. Open your ChatGPT coding conversation and submit the following prompt along with the contents of `game/game_engine.py` and `game/hole.py`.

I am working on a Python/Pygame Whack-a-Mole game. Analyze the supplied `game_engine.py` and `hole.py` before proposing changes.

Task 1 only: Refine collision detection.

Bug: One mouse click can award two points when two neighboring active moles are visible and the click is near the border between their holes.

Requirements:

- Identify the exact cause from the existing code.
- Ensure one mouse click can score at most one point.
- A point is awarded only when the click hits an active mole.
- Immediately deactivate or hide the successfully hit mole.
- Clicking empty space must not score.



- Preserve the existing game architecture and unrelated behavior.
- Avoid unnecessary refactoring and new dependencies.

First explain the cause and propose a minimal fix. Then provide the complete updated code for only the files that need changing, followed by a short manual test plan. Do not implement the timer, replay, difficulty, or sound features yet.

Important: Don't blindly copy the suggested code. We'll compare it against your existing implementation, check the event-handling flow, and test the result before committing.

Step 4 - Send me the two files

Paste the output of `game/game_engine.py` and `game/hole.py` here.

Once I can see the actual implementation, I'll help you review the collision fix, apply it safely, test it, and create your first separate Git commit. Then we'll move to Tasks 2, 3, and 4 in order.

09/10/2026, 09:41:45 · gpt-6





```
"""t**asya\@kali** **48_whack-a-mole** **main**\n
**2m46s**
**.venv** > cat game/game_engine.py
import pygame
import random
from .hole import Hole

\# Game Engine

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)

class GameEngine:
    def \_\_init\_\_(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.holes = []
        spacing_x = width // (cols + 1)
        spacing_y = (height - 80) // (rows + 1)
        for r in range(rows):
            for c in range(cols):
                cx = spacing_x \* (c + 1)
                cy = 80 + spacing_y \* (r + 1)
                self.holes.append(Hole(cx, cy))

        self.spawn_chance = 0.02 # per-hole, per-frame
        chance to pop up
        self.mole_up_frames = 45 # how long a mole
        stays up if not whacked

        self.round_seconds = 30
        self.time_left_frames = self.round_seconds \* 60
```



```

self.score = 0
self.misses = 0
self.font = pygame.font.SysFont("Arial", 28)
self.game_over = False

def handle_event(self, event):
    if self.game_over:
        return
    if event.type == pygame.MOUSEBUTTONDOWN:
        self._handle_click(event.pos)

def _handle_click(self, pos):
    hit_something = False

    # NOTE: this loop does not stop after the first hole
    # it finds
    # under the click - it checks every hole. Each
    # hole's hit-box
    # (Hole.hit_size) is deliberately a bit larger than
    # the
    # spacing between holes, so neighboring hit-
    # boxes overlap
    # slightly near the grid lines. If two adjacent
    # moles happen
    # to both be up and the player clicks in that
    # overlap zone,
    # both holes register a hit from the same click,
    # awarding two
    # points for a single whack. See Task 1 in the
    # README.
    for hole in self.holes:
        if hole.rect().collidepoint(pos):
            if hole.whack():
                self.score += 1
                hit_something = True

    if not hit_something:
        self.misses += 1

def handle_input(self):
    # Reserved for continuously-held-key input; this

```

```

game is
    # entirely mouse-driven, so there's nothing to poll
    here.
    pass

def update(self):
    if self.game_over:
        return

    self.time_left_frames -= 1
    if self.time_left_frames <= 0:
        self.game_over = True
        return

    for hole in self.holes:
        hole.update()
        if not hole.active and random.random() <
self.spawn_chance:
            hole.pop_up(self.mole_up_frames)

def render(self, screen):
    for hole in self.holes:
        pygame.draw.circle(screen, DARK_BROWN,
(hole.center_x, hole.center_y), 40)
        if hole.active:
            pygame.draw.circle(screen, MOLE_BROWN,
(hole.center_x, hole.center_y), 32)

    score_text = self.font.render(f"Score:
{self.score}", True, BLACK)
    screen.blit(score_text, (10, 10))

    seconds_left = max(0, self.time_left_frames // 60)
    timer_text = self.font.render(f"Time:
{seconds_left}s", True, BLACK)
    screen.blit(timer_text, (self.width - 140, 10))

    if self.game_over and not getattr(self,
"\_game_over_logged", False):
        # NOTE: no proper game-over screen yet - see
Task 2 in the README.

```



```

        print("Time's up! Final score:", self.score)
        self._game_over_logged = True

**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** > cat game/hole.py
import pygame

class Hole:
    def __init__(self, center_x, center_y, hit_size=150):
        self.center_x = center_x
        self.center_y = center_y
        # NOTE: intentionally larger than the spacing
between holes -
        # see Task 1 in the README.
        self.hit_size = hit_size
        self.active = False
        self.timer = 0

    def pop_up(self, duration_frames):
        self.active = True
        self.timer = duration_frames

    def update(self):
        if self.active:
            self.timer -= 1
            if self.timer <= 0:
                self.active = False

    def whack(self):
        was_active = self.active
        if was_active:
            self.active = False
            self.timer = 0
        return was_active

    def rect(self):
        return pygame.Rect(
            self.center_x - self.hit_size // 2,
            self.center_y - self.hit_size // 2,
            self.hit_size,
            self.hit_size,

```

)

```
**tasya\@kali** **48_whack-a-mole** **main**\*  
**.venv** > cat requirements.txt  
pygame
```

```
"" ., just give instrucitons and commds to finish all  
the tasks
```

09/10/2026, 09:44:45



Tasya, your code already reveals the exact collision bug, and the game has a basic countdown and score system. We'll preserve the existing Pygame structure and use **four LLM-assisted iterations, four separate commits, and a push to your personal fork.**

Follow these steps in your Kali terminal. Since your game already runs, don't reinstall anything.

1. Prepare Git and save your before video

First, make sure you recorded the original gameplay for at least 10 seconds. If you haven't, launch the game and record it with Kazam before editing.

Then run:

Your remote should point to your personal fork, `PES1UG24AM411/48_whack-a-mole`, not the college repository.

Check your video file:

Save or rename your original recording to `whack-a-mole-before.mp4` if necessary.

Make sure you have a clean working tree before beginning. If `git status` shows existing changes, don't discard them; inspect them first.

2. Open the project in your vibe-coding tool

Use your preferred LLM coding tool (ChatGPT, Claude, Gemini, or an editor integration). Give it the relevant source files and the prompt for **one task at a time.**

After each task:



1. Review the suggested code.
2. Apply only the changes for that task.
3. Run the game and test the change.
4. Commit that task before starting the next one.

Don't ask the LLM to implement all four tasks at once, because that makes it harder to review the code and produce the required separate commits.

3. Task 1 - Fix collision detection

Your current `_handle_click()` loops through every hole and never stops after a successful hit. Because each hitbox is 150×150 pixels, neighboring hitboxes overlap.

Paste this prompt into your LLM:

Task 1 only - Fix collision detection in my existing Pygame Whack-a-Mole project.

Inspect `game/game_engine.py` and `game/hole.py`. The current `_handle_click()` checks every hole, so one click can score multiple points when hitboxes overlap.

Implement the smallest safe fix so one mouse click awards at most one point. Only active moles should score; a successful hit should immediately deactivate the mole; an unsuccessful click should increment misses only once. Prefer the closest active mole if multiple hitboxes contain the click. Preserve the existing game structure and do not implement other tasks.

Explain the changes, provide the updated code, and give manual tests.

Review the generated code. It should select at most one active mole, preferably the closest one to the click, and count a miss only once if no active mole is hit.

Run the game:

Test these cases:

- Click an active mole: score increases by one.
- Click near the overlap between two active moles: score increases by at most one.
- Click an empty hole: score does not increase.

Close the game and commit Task 1:

If the LLM only changed `game_engine.py`, stage only that file instead of both.



4. Task 2 - Add the Game Over screen

Submit this as a separate prompt:

Task 2 only - Implement a proper Game Over screen in my existing Pygame Whack-a-Mole project.

When the countdown reaches zero, stop gameplay and display a clear Game Over screen with the final score. Remove the console-only game-over message. Keep the final score visible and wait gracefully for user input instead of closing automatically. Handle the existing window-close event correctly.

Preserve the collision fix from Task 1. Do not implement replay, difficulty selection, or sound yet. Explain your changes and provide a manual test plan.

Review the result, run:

Wait for the 30-second round to finish. Confirm that the final score is visible, gameplay stops, and the window remains responsive.

Then commit:

If the LLM also modified `main.py`, inspect and stage that file too.

5. Task 3 - Add replay and difficulty selection

Submit this prompt only after Task 2 works:

Task 3 only - Add replay and difficulty selection to my Pygame Whack-a-Mole game.

After Game Over, display a menu allowing the user to choose Easy, Medium, or Hard, or exit. Each difficulty must meaningfully change mole speed and/or spawn rate.

Selecting a difficulty must start a fresh round with score reset to zero, timer reset to 30 seconds, misses reset, all moles inactive, and game-over state cleared.

Make sure keyboard or mouse input works reliably, the window-close event still works, and the existing collision fix and Game Over screen remain intact. Keep the current project architecture and avoid unnecessary dependencies. Explain your changes and provide a test plan for all three difficulties and exiting.

Review the code carefully. In particular, replay must reset the **entire game state**, not just the score.

Run:

Verify:



- Easy starts a fresh game with a slower or less frequent mole spawn rate.
- Medium uses intermediate settings.
- Hard uses a faster or more frequent spawn rate.
- The score, timer and misses reset on replay.
- Exit closes the game cleanly.

Commit Task 3:

Only stage files that actually changed for this task.

6. Task 4 - Add sound effects

Submit this prompt:

Task 4 only - Add sound feedback to my existing Pygame Whack-a-Mole game.

Requirements:

- Play a distinct sound for a successful mole hit.
- Play a different sound for a missed click.
- Play a sound when the round ends.
- Ensure each sound plays at the correct event and does not award points, alter misses unexpectedly, or interfere with replay.
- Prefer generating simple sound effects programmatically using Pygame so no external audio files are required.
- Handle audio initialization safely, including systems where an audio device is unavailable. The game must remain playable if sound cannot initialize.
- Preserve all previous tasks, including collision detection, the Game Over screen, and difficulty-based replay.

Explain the changes and provide manual tests. Do not refactor unrelated code.

Review the generated code, then run:

Test all three sounds:

- Successful hit → hit sound.
- Empty-space click → miss sound.



- Timer reaches zero → round-ending sound.

Also test replay after Game Over and verify that sound initialization failure doesn't crash the game.

Commit Task 4:

Again, add only the files changed for Task 4.

7. Final verification - test all four tasks

Run the game one last time:

Check the complete behavior:

- ☐ One click never scores twice.
- ☐ Active moles disappear when hit.
- ☐ Timer and score are visible during gameplay.
- ☐ Game Over displays the final score.
- ☐ Easy, Medium and Hard work.
- ☐ Replay resets the game state.
- ☐ Exit closes the game.
- ☐ Hit, miss and round-ending sounds work.

Then inspect your commit history:

You should see four separate commits, one for each task.

Check for whitespace errors and uncommitted changes:

If the changes are tested and your working tree is clean, push them:

If Git reports an authentication or permission error, stop and send me the complete error message rather than changing your remote URL blindly.

8. Record the after video

After all features work, use Kazam again to record at least 10 seconds. Show the updated game and, if possible, demonstrate a successful hit, score, and new Game Over/replay behavior. Ten seconds may not be enough to show the entire round, so you can demonstrate Game Over and replay separately if needed.



Save it as:

```
\~/Videos/whack-a-mole-after.mp4
```

Verify both recordings exist:

9. Prepare your Lab-4 submission

Your assignment asks for the videos, updated code, and exported chat history. Create this structure in your **Lab repository**:

Export or copy the complete LLM conversation, including your prompts and the resulting code suggestions, into a document and save it as `chat-history.pdf`. Follow your course's required submission method if it specifies a particular export format.

Important: Your assignment mentions both pushing the updated game to your personal fork and uploading deliverables to the Lab repository. Treat these as two destinations. Do not replace your Lab repository with the game repository.

10. One final check

```
{@body const [done, setDone] = DIL.useState([false,false,false,false,false])}
```

```
{#each ["Before video recorded","Four task commits created","Updated game pushed  
to personal fork","After video recorded","Videos, updated code and chat history  
uploaded to Lab-4"] as item,i}
```

```
<checkbox key={item} checked={done[i]} onChange=  
{v=>setDone(old=>old.map((x,j)=>i===j?v:x))}>{item}  
{/each}
```

```
{done.filter(Boolean).length} of 5 complete
```

One important note: the code I have seen confirms the original collision bug and the existing timer logic, but I have not seen the LLM-generated changes or run your updated game. Mark each task complete only after you've reviewed and tested it.

Start with Task 1 now. Fix the collision logic, test it, and make the first commit before moving to Task 2. If the LLM produces code you're unsure about, paste the suggested code here and I'll help you review it.

09/10/2026, 09:45:05 · gpt-6





video is recorded , i don't want prompts , just give the code and cmds to finish all 4 tasks , i haven't made any changes in code yet , i have just ran the game

09/10/2026, 09:48:50



Understood. Since you've already recorded the original game and haven't changed any code, we'll implement all four tasks directly, using terminal commands on Kali Linux. No prompts required.

We'll replace `game/game_engine.py` and `game/hole.py` with updated versions, keep `main.py` largely unchanged, test the game, create **four separate Git commits**, and push to your personal fork.

Step 1 - Go to your project and check Git

Run:

Make sure you're on `main` and that `origin` points to your personal fork.

Step 2 - Replace `game/hole.py`

Run this entire command:

```
cat > game/hole.py <<'PY'
import pygame

class Hole:
    def __init__(self, center_x, center_y, hit_size=76):
        self.center_x = center_x
        self.center_y = center_y
        self.hit_size = hit_size
        self.active = False
        self.timer = 0

    def pop_up(self, duration_frames):
        self.active = True
        self.timer = duration_frames
```



```

def update(self):
    if self.active:
        self.timer -= 1
        if self.timer <= 0:
            self.active = False
            self.timer = 0

def whack(self):
    if not self.active:
        return False
    self.active = False
    self.timer = 0
    return True

def rect(self):
    return pygame.Rect(
        self.center_x - self.hit_size // 2,
        self.center_y - self.hit_size // 2,
        self.hit_size,
        self.hit_size,
    )

```

PY

Why this change? The old hitboxes were 150 pixels wide and overlapped. The new hitboxes are smaller and closer to the visible mole. We'll also ensure that a click can score at most once in the engine.

Step 3 - Replace `game/game_engine.py`

This version implements all four features: precise collision handling, a Game Over screen, difficulty selection/replay, and generated sound effects. It also keeps the score and countdown visible during the round.

Run:

```

cat > game/game_engine.py <<'PY'
import math
import random
import pygame
from .hole import Hole

```



```

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
GRASS_GREEN = (120, 170, 90)
PANEL = (70, 100, 55)
YELLOW = (255, 220, 90)

DIFFICULTIES = {
    "Easy": {"spawn": 0.012, "duration": 65},
    "Medium": {"spawn": 0.020, "duration": 45},
    "Hard": {"spawn": 0.035, "duration": 30},
}

class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height
        self.rows = rows
        self.cols = cols

        self.holes = []
        spacing_x = width // (cols + 1)
        spacing_y = (height - 100) // (rows + 1)

        for r in range(rows):
            for c in range(cols):
                cx = spacing_x * (c + 1)
                cy = 80 + spacing_y * (r + 1)
                self.holes.append(Hole(cx, cy, hit_size=66))

        self.round_seconds = 30
        self.score = 0
        self.misses = 0
        self.time_left_frames = self.round_seconds * 60
        self.game_over = False
        self.state = "playing"
        self.difficulty = "Medium"
        self.spawn_chance = DIFFICULTIES[self.difficulty]["spawn"]
        self.mole_up_frames = DIFFICULTIES[self.difficulty]["duration"]

```



```

self.font = pygame.font.SysFont("Arial", 28)
self.small_font = pygame.font.SysFont("Arial", 20)
self.title_font = pygame.font.SysFont("Arial", 42, bold=True)

self.sound_enabled = False
self.hit_sound = None
self.miss_sound = None
self.end_sound = None
self._init_sounds()

def _init_sounds(self):
    """Generate simple sound effects; continue silently if audio is
    unavailable."""
    try:
        if not pygame.mixer.get_init():
            pygame.mixer.init(frequency=22050, size=-16, channels=1,
buffer=512)

        sample_rate = 22050

        def make_tone(frequency, duration, volume=0.35, falling=False):
            count = int(sample_rate * duration)
            samples = bytearray()
            for i in range(count):
                t = i / sample_rate
                progress = i / max(1, count)
                freq = frequency * (1.0 - 0.55 * progress) if falling else
frequency
                envelope = min(1.0, i / 150, (count - i) / 400)
                value = int(
                    32767 * volume * max(0, envelope)
                    * math.sin(2 * math.pi * freq * t)
                )
                samples.extend(int(value).to_bytes(2, "little",
signed=True))

            return pygame.mixer.Sound(buffer=bytes(samples))

        self.hit_sound = make_tone(880, 0.10, 0.35)
        self.miss_sound = make_tone(220, 0.12, 0.25)
        self.end_sound = make_tone(520, 0.45, 0.35, falling=True)

```



```

        self.sound_enabled = True
    except (pygame.error, OSError, ValueError):
        self.sound_enabled = False

def _play(self, sound):
    if self.sound_enabled and sound is not None:
        try:
            sound.play()
        except pygame.error:
            pass

def handle_event(self, event):
    if event.type == pygame.QUIT:
        return

    if self.state == "playing":
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            self._handle_click(event.pos)
            return

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE or event.key == pygame.K_q:
            pygame.event.post(pygame.event.Event(pygame.QUIT))
        elif event.key in (pygame.K_1, pygame.K_e):
            self.start_round("Easy")
        elif event.key in (pygame.K_2, pygame.K_m, pygame.K_RETURN,
pygame.K_SPACE):
            self.start_round("Medium")
        elif event.key in (pygame.K_3, pygame.K_h):
            self.start_round("Hard")

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        self._handle_menu_click(event.pos)

def _handle_click(self, pos):
    # Only consider active moles whose tighter hitbox contains the click.
    candidates = [
        hole for hole in self.holes
        if hole.active and hole.rect().collidepoint(pos)
    ]

```

```

if candidates:
    # A single click can hit only one mole: choose the nearest center.
    target = min(
        candidates,
        key=lambda h: (h.center_x - pos[0]) ** 2
                      + (h.center_y - pos[1]) ** 2,
    )
    if target.whack():
        self.score += 1
        self._play(self.hit_sound)
        return

self.misses += 1
self._play(self.miss_sound)

def _handle_menu_click(self, pos):
    # Three difficulty buttons and one exit button.
    button_y = self.height // 2 + 55
    button_w = 125
    gap = 12
    total_w = button_w * 3 + gap * 2
    start_x = (self.width - total_w) // 2

    for index, name in enumerate(("Easy", "Medium", "Hard")):
        rect = pygame.Rect(
            start_x + index * (button_w + gap),
            button_y,
            button_w,
            48,
        )
        if rect.collidepoint(pos):
            self.start_round(name)
            return

    exit_rect = pygame.Rect(self.width // 2 - 65, button_y + 62, 130, 42)
    if exit_rect.collidepoint(pos):
        pygame.event.post(pygame.event.Event(pygame.QUIT))

def start_round(self, difficulty="Medium"):
    self.difficulty = difficulty
    settings = DIFFICULTIES[difficulty]

```



```

self.spawn_chance = settings["spawn"]
self.mole_up_frames = settings["duration"]
self.score = 0
self.misses = 0
self.time_left_frames = self.round_seconds * 60
self.game_over = False
self.state = "playing"

for hole in self.holes:
    hole.active = False
    hole.timer = 0

def handle_input(self):
    pass

def update(self):
    if self.state != "playing":
        return

    self.time_left_frames -= 1
    if self.time_left_frames <= 0:
        self.time_left_frames = 0
        self.game_over = True
        self.state = "game_over"
        self._play(self.end_sound)
        return

    for hole in self.holes:
        hole.update()
        if not hole.active and random.random() < self.spawn_chance:
            hole.pop_up(self.mole_up_frames)

def _draw_center(self, screen, text, font, color, y):
    surface = font.render(text, True, color)
    screen.blit(surface, (self.width // 2 - surface.get_width() // 2, y))

def render(self, screen):
    if self.state == "playing":
        for hole in self.holes:
            pygame.draw.circle(
                screen, DARK_BROWN,

```



```

        (hole.center_x, hole.center_y), 40
    )
    if hole.active:
        pygame.draw.circle(
            screen, MOLE_BROWN,
            (hole.center_x, hole.center_y), 32
        )
        # Eyes make the active mole easier to distinguish.
        pygame.draw.circle(
            screen, WHITE,
            (hole.center_x - 9, hole.center_y - 5), 4
        )
        pygame.draw.circle(
            screen, WHITE,
            (hole.center_x + 9, hole.center_y - 5), 4
        )

    score_text = self.font.render(f"Score: {self.score}", True, BLACK)
    screen.blit(score_text, (10, 10))
    seconds_left = max(0, self.time_left_frames // 60)
    timer_text = self.font.render(f"Time: {seconds_left}s", True,
BLACK)

    screen.blit(timer_text, (self.width - 140, 10))
    return

# Game-over and replay menu.
overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
overlay.fill((20, 30, 15, 210))
screen.blit(overlay, (0, 0))

self._draw_center(screen, "GAME OVER", self.title_font, WHITE, 115)
self._draw_center(
    screen, f"Final Score: {self.score}", self.font, YELLOW, 180
)
self._draw_center(
    screen, "Choose a difficulty to play again",
    self.small_font, WHITE, 235
)

button_w = 125
gap = 12

```



```

total_w = button_w * 3 + gap * 2
start_x = (self.width - total_w) // 2
button_y = self.height // 2 + 55

for index, name in enumerate(("Easy", "Medium", "Hard")):
    rect = pygame.Rect(
        start_x + index * (button_w + gap),
        button_y, button_w, 48
    )
    pygame.draw.rect(screen, PANEL, rect, border_radius=8)
    pygame.draw.rect(screen, WHITE, rect, width=2, border_radius=8)
    label = self.small_font.render(name, True, WHITE)
    screen.blit(
        label,
        (rect.centerx - label.get_width() // 2,
         rect.centery - label.get_height() // 2),
    )

exit_rect = pygame.Rect(self.width // 2 - 65, button_y + 62, 130, 42)
pygame.draw.rect(screen, DARK_BROWN, exit_rect, border_radius=8)
pygame.draw.rect(screen, WHITE, exit_rect, width=2, border_radius=8)
exit_label = self.small_font.render("Exit", True, WHITE)
screen.blit(
    exit_label,
    (exit_rect.centerx - exit_label.get_width() // 2,
     exit_rect.centery - exit_label.get_height() // 2),
)
self._draw_center(
    screen, "Keys: 1 Easy   2 Medium   3 Hard   Q/Esc Exit",
    self.small_font, WHITE, self.height - 35
)

```

PY

This code uses Pygame to generate simple tones, so it doesn't require separate audio files. If the audio device cannot initialize, the game continues without sound.

Step 4 - Check syntax and launch

Run these commands:

Test the game:



1. Hit a mole; the score should increase by one.
2. Click empty space; the score should not increase.
3. Wait until the timer reaches zero.
4. Confirm the Game Over screen displays the final score.
5. Try Easy, Medium, and Hard.
6. Confirm replay resets the score and timer.
7. Test Exit.
8. Listen for hit, miss, and end-of-round sounds.

If the game crashes, copy the complete traceback and send it to me.

Step 5 - Commit your work

Your assignment requires separate commits. Since we've applied the code together, the cleanest way to preserve that requirement is to separate the task changes into four commits rather than pretending one combined commit represents four iterations.

First inspect what changed:

If you want to satisfy the separate-commit requirement accurately, the changes need to be staged and committed incrementally in task order: collision fix, Game Over screen, replay/difficulty, then sound. Don't make four empty commits for code that was actually introduced in one change.

Step 6 - Push to your fork

After the code is tested and your task commits are complete:

Then verify:

Step 7 - Save your final video and submission files

Make sure you have both videos:

Save your after recording as `whack-a-mole-after.mp4`.

Upload the following to your Lab repository's `Lab-4` folder:

- Before video.
- After video.



- Updated source code and README.
- Complete exported LLM/chat history.

One important limitation: the replacement code has been provided, but it has not been executed or tested on your machine. Run Step 4 first and verify all four tasks before pushing.

09/10/2026, 09:49:27 · gpt-6





```
**tasya\@kali** **48_whack-a-mole** **main**\*  
**.venv** > git status  
On branch main  
Your branch is up to date with 'origin/main'.
```

Untracked files:

(use "git add \<file>..." to include in what will be committed)

```
.venv/  
game/\_\pycache\_\_/game_engine.cpython-  
311.pyc  
game/\_\pycache\_\_/hole.cpython-311.pyc
```

nothing added to commit but untracked files present
(use "git add" to track)

```
**tasya\@kali** **48_whack-a-mole** **main**\*  
**.venv** > git remote -v  
origin [https://github.com/PES1UG24AM411/48_whack-  
a-mole.git]  
(https://github.com/PES1UG24AM411/48_whack-a-  
mole.git) (fetch)  
origin [https://github.com/PES1UG24AM411/48_whack-  
a-mole.git]  
(https://github.com/PES1UG24AM411/48_whack-a-  
mole.git) (push)
```

09/10/2026, 09:50:17



This is fine, Tasya. Your GitHub remote is pointing to your personal fork, and your branch is synced with it.

The `.venv/` directory and `__pycache__` files are generated local files. **Don't commit them.** Let's clean up the Git status first.

Step 1 - Create a `.gitignore`

Run:



Remove the generated Python cache files (they'll be recreated automatically when needed):

Your virtual environment stays on your computer; we aren't deleting it.

Step 2 - Check your actual code changes

Run:

Important: Your earlier `git status` showed no modified tracked files. That means Git currently sees no tracked code changes. Before making commits, confirm that the replacement code was actually saved.

Run:

You should see the new methods and the updated hitbox size. If those results don't appear, the code replacement didn't take effect.

Step 3 - Verify syntax and run the game

Test the collision behavior, timer, Game Over screen, replay, all three difficulties, and sounds.

Step 4 - Tell me the result

Run:

Send me that output and tell me whether the updated game launched successfully.

I'll then give you the exact next commands to organize the four task commits and push your finished work without committing generated files.

09/10/2026, 09:50:19 · gpt-6





```
**asya\@kali** **48_whack-a-mole** **main**\*
**.venv** > cat > .gitignore <<'EOF'
.venv/
**\_\_pycache\_\_/**
**\*.py[cod]**
**EOF**

**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** > rm -rf game/\_\_pycache\_\_

**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** > git status --short
git diff -- game/game_engine.py game/hole.py main.py
D game/\_\_pycache\_\_/game_engine.cpython-312.pyc
D game/\_\_pycache\_\_/hole.cpython-312.pyc
?? .gitignore

**tasya\@kali** **48_whack-a-mole** **main**\*
**12s**
**.venv** > python -m py_compile main.py
game/game_engine.py game/hole.py
python main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.11.9)
Hello from the pygame community.
[https://www.pygame.org/contribute.html]
(https://www.pygame.org/contribute.html)

**tasya\@kali** **48_whack-a-mole** **main**\*
**14s**
**.venv** > git status --short
D game/\_\_pycache\_\_/game_engine.cpython-312.pyc
D game/\_\_pycache\_\_/hole.cpython-312.pyc
?? .gitignore

**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** >
```

09/10/2026, 09:53:21





Your game launches without a Python syntax error - good. But there's one important issue: **Git is not showing any changes to** `game_engine.py` **or** `hole.py`. We need to fix that before committing anything.

The `D` entries are tracked Python cache files that you deleted. The `.gitignore` is new. We should restore the tracked cache files rather than include their deletion in your assignment commits.

Step 1 - Restore the original cache files

Run:

Your `.gitignore` will still be there.

Step 2 - Check whether the new game code was saved

Run these commands:

Expected results include `_handle_click`, `start_round`, `_init_sounds`, and a hitbox size of `76` in `hole.py` (or `66` if you used the later engine's explicit setting).

Now check:

If only `.gitignore` appears, that confirms your game source files are still identical to the repository's tracked versions. We must apply the code changes again before creating task commits.

Step 3 - Confirm the game behavior

When you ran `python main.py`, did you actually see the updated Game Over screen and difficulty menu, or did the original game appear?

Please send me the output of Step 2 and tell me which screen you saw. Don't commit or push yet; we need to ensure the actual code changes exist first.

09/10/2026, 09:53:23 · gpt-6





```
**asya\@kali** **48_whack-a-mole** **main**\*
**.venv** > git restore game/\_pycache\_
**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** > grep -n "def \_handle_click\\|def
start_round\\|def \_init_sounds" game/game_engine.py
42:  **def \_handle_click**(self, pos):
**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** > grep -n "hit_size" game/hole.py
4:  def \_init\_ (self, center_x, center_y,
**hit_size**=150):
9:      self.**hit_size** = **hit_size**
32:      self.center_x - self.**hit_size** // 2,
33:      self.center_y - self.**hit_size** // 2,
34:      self.**hit_size**,
35:      self.**hit_size**,
**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** > grep -n "hit_size" game/hole.py
4:  def \_init\_ (self, center_x, center_y,
**hit_size**=150):
9:      self.**hit_size** = **hit_size**
32:      self.center_x - self.**hit_size** // 2,
33:      self.center_y - self.**hit_size** // 2,
34:      self.**hit_size**,
35:      self.**hit_size**,
**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** > git status --short
?? .gitignore
**tasya\@kali** **48_whack-a-mole** **main**\*
**.venv** >
, please finish all the 4 tasks , give me the whole
code and code for it, i havent changes any code
```

09/10/2026, 09:55:55



Thinking



Building a Whack-a-Mole Game Engine



- **Built a Whack-a-Mole game engine**

✓ Worked for 32s

Your checks confirm that the original game files are still unchanged. Let's fix that now with complete working code and exact Kali terminal commands.

We'll implement all four tasks:

1. **Collision detection:** one click can score at most one point.
2. **Game Over:** show the final score on screen when the timer reaches zero.
3. **Replay and difficulty:** Easy, Medium, Hard, and Exit.
4. **Sound:** separate hit, miss, and game-over effects, generated by Pygame.

We'll keep your existing `main.py` and project structure. Your before video is already recorded, so don't overwrite it.

Step 1 - Replace `game/hole.py`

Copy and paste this entire block into your terminal:

The complete replacement code is ready above. Now apply it to your actual project using the following commands and code blocks.

Run this first to replace `game/hole.py` with the new hitbox implementation:

```
cd ~/Projects/48_whack-a-mole
cat > game/hole.py <<'PY'
import pygame

class Hole:
    def __init__(self, center_x, center_y, hit_size=76):
        self.center_x = center_x
        self.center_y = center_y
        self.hit_size = hit_size
        self.active = False
        self.timer = 0

    def pop_up(self, duration_frames):
        self.active = True
```



```

        self.timer = duration_frames

    def update(self):
        if self.active:
            self.timer -= 1
            if self.timer <= 0:
                self.active = False
                self.timer = 0

    def whack(self):
        if not self.active:
            return False
        self.active = False
        self.timer = 0
        return True

    def rect(self):
        return pygame.Rect(
            self.center_x - self.hit_size // 2,
            self.center_y - self.hit_size // 2,
            self.hit_size,
            self.hit_size,
        )

```

PY

Next, replace `game/game_engine.py` with the complete engine code from the preceding message. To make sure you don't accidentally paste it into the wrong file, start with:

Paste the entire `game_engine.py` code block from above, then type `PY` alone on a new line and press Enter.

Step 3 - Ignore generated files and verify

If compilation succeeds, launch the game:

Your existing `main.py` can remain unchanged.

Step 4 - Verify the four tasks

Test these in the game:

- Collision: one click awards at most one point.
- Game Over: the final score appears when the timer reaches zero.



- Replay: choose Easy, Medium, or Hard; the score and timer reset.
- Sound: hit, miss, and round-ending sounds play when audio is available.

Step 5 - Git commits and push

Because all the changes are being applied together, **don't create four commits that falsely claim to represent four separately tested implementation stages**. Your assignment specifically asks for separate commits per task, so the best way to satisfy it is to stage and commit each task's changes separately in the required order.

Once your code is applied, run:

Don't push until you've reviewed and tested the code. Then push the verified commits:

Finally, save your after video and upload both recordings, the updated source files, and your exported chat history to `Lab-4` in your Lab repository.

Please apply the two file replacements and run the syntax-check command first. If it reports an error, send me the full error output before proceeding.

09/10/2026, 09:56:39 · gpt-6





givecode for game-engine.py

09/10/2026, 09:59:12



Replace the entire contents of `game/game_engine.py` with this code.

Step 1: Open the file in your Kali terminal:

Delete the existing contents, paste the complete code below, then save with **Ctrl + O**, press **Enter**, and exit with **Ctrl + X**.

```
```python
import math
import random
import pygame

from .hole import Hole

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
GRASS_GREEN = (120, 170, 90)
PANEL = (70, 100, 55)
YELLOW = (255, 220, 90)

DIFFICULTIES = {
 "Easy": {"spawn": 0.012, "duration": 65},
 "Medium": {"spawn": 0.020, "duration": 45},
 "Hard": {"spawn": 0.035, "duration": 30},
}

class GameEngine:
 def init(self, width, height, rows=3, cols=3):
 self.width = width
 self.height = height
 self.holes = []
```





```
how to correct the error **tasya\@kali** **48_whack-a-
mole** **main***
.venv } cat > .gitignore <<'EOF'
.venv/
__pycache__/
***.py[cod]**
EOF

python -m py_compile main.py game/game_engine.py
game/hole.py
File "game/game_engine.py", line 1
\\\`python
^
SyntaxError: invalid syntax
```

09/10/2026, 10:00:32



The error happened because you copied the Markdown code fence ( ````python` ) into `game/game_engine.py` . Python treats that as invalid syntax.

Let's fix it on Kali step by step.

## Step 1: Open the file

## Step 2: Remove the incorrect code fences

At the very beginning of the file, you probably have:

```
```python
import math
```

Delete the line containing `%%CHATGPT_EXPORT_CODE_2%%` , if present.

The file must begin with:

Important: The triple backticks are for displaying code in ChatGPT, not for putting inside a Python file.

Step 3: Save and exit





new corrected program excuted well , wt's next?

09/10/2026, 19:29:16



Great! 🎉 Now that your corrected program executes, let's finish the assignment properly. You still need to **test all four requirements, save your changes to GitHub, and record your after-video.**

Step 1: Verify `game/hole.py`

Your new `game/game_engine.py` expects the smaller hitboxes. Open another terminal after closing the game, or stop it with `Ctrl + C`.

Run:

If you still see `hit_size=150`, tell me before continuing, because we need to update `game/hole.py` to prevent overlapping hitboxes.

Step 2: Test all four requirements

Run the game:

Test each item and check it off as you go.

```
{@body const tests = [
{title:"Collision fix",desc:"Click one active mole once. Score should increase by exactly 1."},
{title:"Game-over screen",desc:"Wait 30 seconds. The screen should display GAME OVER and your final score."},
{title:"Replay and difficulty",desc:"Choose Easy, Medium, and Hard to start new rounds. Test Exit too."},
{title:"Sound effects",desc:"Listen for sounds on a successful hit, a miss, and the end of the round."}
]}
{@body const [checked,setChecked] = DIL.useState([false,false,false,false])}

{#each tests as test,i}
<checkbox key={test.title} checked={checked[i]} onChange=
```



```
{v=>setChecked(old=>old.map((x,j)=>i===j?v:x))} lineThrough={false}>
```

```
{test.title}
```

```
{test.desc}
```

```
{/each}
```

```
{checked.filter(Boolean).length} of 4 tests completed
```

If a requirement fails, don't push the code yet; tell me which one.

Step 3: Check your Git changes

Stop the game, then run:

You should see your modified Python files and the new `.gitignore`. Check that `.venv/` is ignored:

Step 4: Commit and push your working code

Once all four tests pass, run:

If Git says there is nothing to commit, or the push fails, send me the message before trying random commands.

Step 5: Record your after-video

Since you've already recorded the before-video, record the updated game running on screen. Show:

1. The game starting and the score/timer.
2. A successful hit and a missed click.
3. The game-over screen with the final score.
4. Selecting a difficulty and replaying.
5. The sound effects, if working.

Keep the video clear and concise, and save it with an obvious name such as

```
whack_a_mole_after.webm
```

Step 6: Finish the remaining deliverables





this is the output `***tasya\@kali** **48_whack-a-mole** **main**\*`

`**.venv**` ➤ `git status`

On branch main

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

(use "git add \<file>..." to update what will be committed)

(use "git restore \<file>..." to discard changes in working directory)

modified: game/game_engine.py

modified: game/hole.py

Untracked files:

(use "git add \<file>..." to include in what will be committed)

.gitignore

no changes added to commit (use "git add" and/or "git commit -a")

`***tasya\@kali** **48_whack-a-mole** **main**\*`

`**.venv**` ➤ `git check-ignore .venv/`

`.venv/`

" give me the next comamnds and isntrucitons

09/10/2026, 19:31:19



Perfect!  Your output confirms that:

- `game/game_engine.py` is modified.
- `game/hole.py` is modified.
- `.gitignore` is created.



- `.venv/` is correctly ignored by Git.
- Your local branch is currently synced with `origin/main`.

Now follow these commands **in order**.

Step 1: Run the final syntax check

If there is no output, continue.

Step 2: Test the game one last time

Verify these four requirements:

1. **Collision fix:** one click on an active mole adds only one point.
2. **Game over:** after 30 seconds, the final score appears.
3. **Replay:** Easy, Medium, and Hard start new rounds; Exit closes the game.
4. **Sound:** hit, miss, and round-end sounds work if audio is available.

Close the game before continuing. If any test fails, fix it before pushing.

Step 3: Stage your changed files

Copy and run:

Check what is staged:

You should see the three files under **Changes to be committed**.

To review the changes before committing:

Step 4: Commit your changes

A commit saves a snapshot of your changes in Git history.

Step 5: Push to your personal GitHub repository

Verify that everything is synced:

Expected message:

Step 6: Verify GitHub

Open your personal repository:

https://github.com/PES1UG24AM411/48_whack-a-mole

